

Assignment 03: Smart Temperature Monitoring

1. Introduzione

Il presente documento descrive la progettazione e lo sviluppo di un sistema IoT per il monitoraggio della temperatura in un ambiente chiuso dotato di finestra. Il sistema è suddiviso in quattro sottosistemi principali:

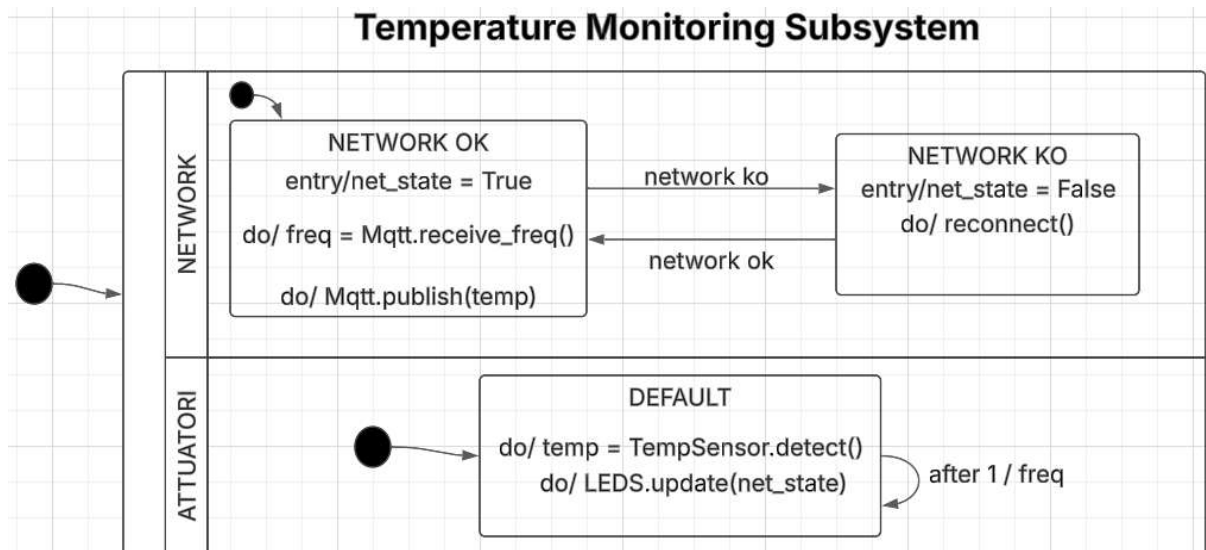
- **Temperature Monitoring Subsystem (TMS)**: responsabile del monitoraggio della temperatura tramite ESP32.
- **Control Unit Subsystem (CUS)**: backend su PC che gestisce la logica del sistema.
- **Window Controller Subsystem (WCS)**: sistema embedded basato su Arduino per il controllo della finestra.
- **Dashboard Subsystem (DSHB)**: interfaccia web per la visualizzazione dei dati e il controllo del sistema.

2. Architettura del Sistema

2.1 Temperature Monitoring Subsystem (TMS)

Il TMS utilizza ESP32 con **FreeRTOS** per la gestione dei task:

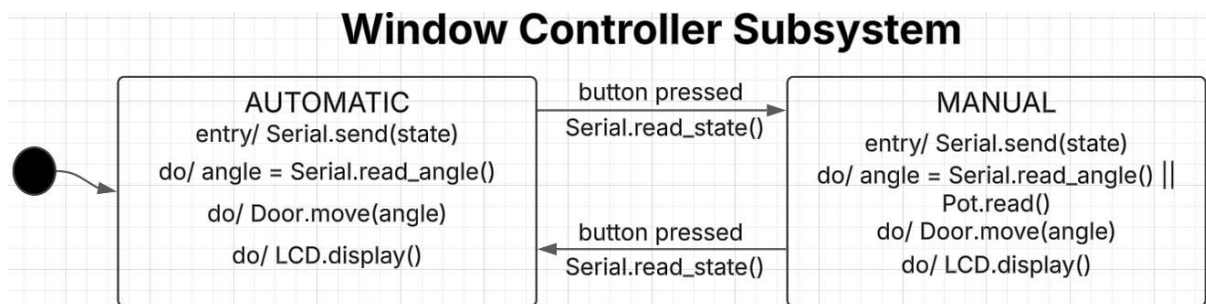
- Utilizza la libreria **PubSubClient** per la comunicazione MQTT con il CUS.
- Due core separati gestiscono rispettivamente gli attuatori e le operazioni di rete.
- L'architettura si basa su multithreading, con l'uso delle API di FreeRTOS per la creazione di **task** e **code**.
- Le code di messaggi garantiscono una comunicazione sicura tra i task, evitando condizioni di race condition nell'accesso ai dati condivisi (frequenza di campionamento, stato di rete, temperatura rilevata).
- La macchina a stati finiti (FSM) prevede due stati: **network_ok** e **network_ko**, a seconda dello stato della rete e dell'invio dei dati.



2.2 Window Controller Subsystem (WCS)

Il WCS è basato su Arduino e gestisce il controllo della finestra tramite:

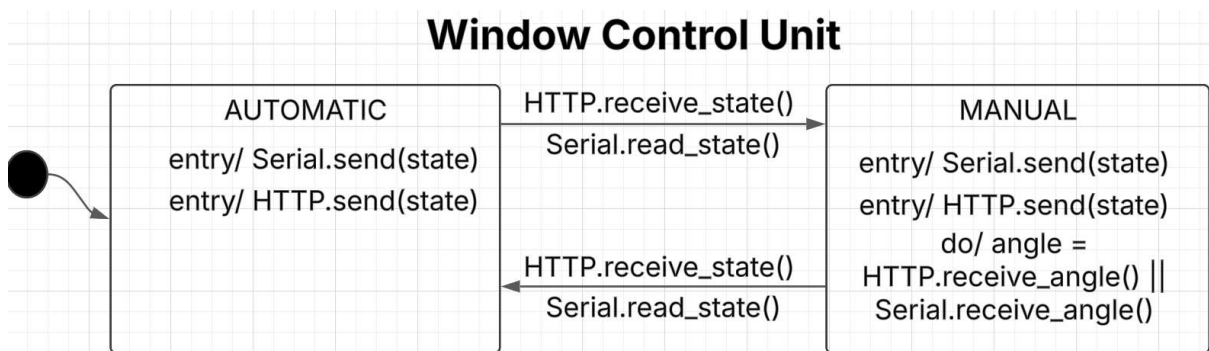
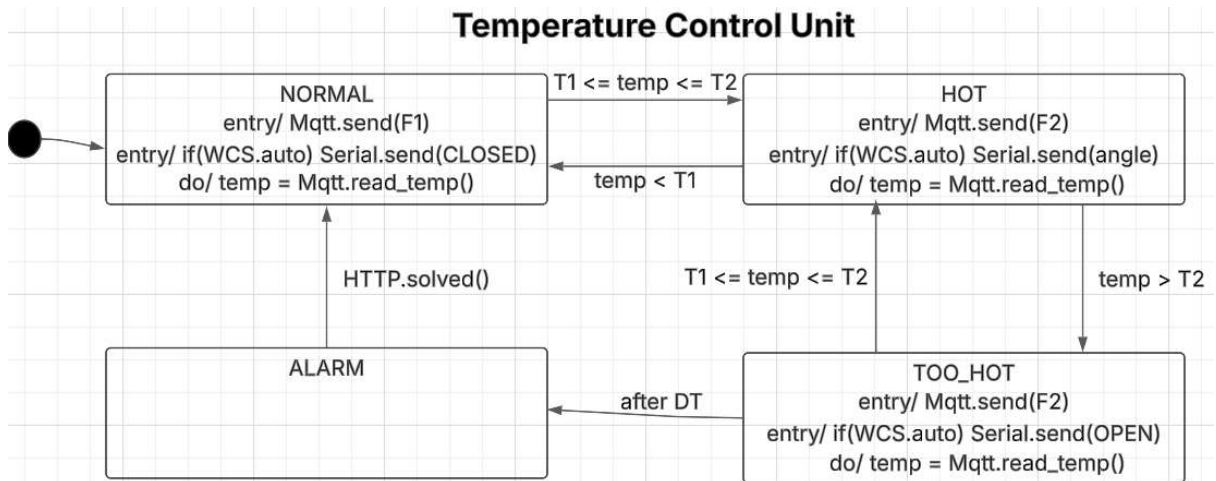
- Servo motore per l'apertura della finestra.
- Potenzimetro per il controllo manuale.
- Pulsante con gestione asincrona mediante interrupt per passare tra modalità manuale e automatica.
- Display LCD per mostrare stato e temperatura.
- Comunicazione con il CUS tramite seriale.



2.3 Control Unit Subsystem (CUS)

Il backend è sviluppato in **Java** utilizzando **Vert.x**:

- Un verticle per ogni sottosistema (**TMS**, **WCS**, **DSHB**) funge da interfaccia per ricezione e invio messaggi.
- I messaggi vengono pubblicati su un **event bus**.
- Un ulteriore verticle **SystemManager** gestisce la logica attraverso due FSM:
 - **tempCU** con stati: **NORMAL**, **HOT**, **TOO_HOT**, **ALARM**.
 - **windowCU** con stati: **AUTOMATIC**, **MANUAL**.



- I messaggi ricevuti vengono processati dalle fsm tempCU e windowCU per determinare le azioni da intraprendere e inviati sull'event bus.
- Canali distinti per temperatura, frequenza, stato della tempCU e stato della windowCU.

2.4 Dashboard Subsystem (DSHB)

La dashboard è una pagina web che permette di:

- Visualizzare lo stato del sistema tramite grafici (temperatura, stato della finestra, modalità operativa).
- Entrare in **modalità manuale** e controllare l'apertura della finestra.
- Gestire lo stato di **allarme**, riportando il sistema alla normalità.
- La comunicazione con il CUS avviene tramite HTTP.

3. Implementazione

L'implementazione è stata suddivisa nei seguenti componenti:

- **ESP32 (TMS)**: lettura del sensore di temperatura e invio dati via MQTT.
- **Arduino UNO (WCS)**: gestione del servo motore e della modalità manuale.
- **Java Vert.x (CUS)**: gestione della logica del sistema e comunicazione tra i componenti.
- **Web App (DSHB)**: interfaccia per il monitoraggio e controllo.

4. Conclusione

Il sistema realizzato soddisfa i requisiti specificati, fornendo un controllo efficace della temperatura e dell'apertura della finestra. L'uso di FSM e di un'architettura basata su event bus garantisce modularità e scalabilità. Possibili miglioramenti futuri includono l'ottimizzazione della gestione degli stati e l'integrazione con altri sensori ambientali.